

Лекция 1. Введение в C++ История языка, стандарты (C++11/14/17/20), основные возможности и сферы применения. Структура программы.

Введение в язык программирования C++

1. Введение

Язык программирования C++ — один из самых мощных, гибких и широко используемых языков программирования общего назначения.

Он сочетает в себе **высокую производительность** (как у C) и **объектно-ориентированные возможности**, которые позволяют создавать сложные, масштабируемые программные системы.

C++ используется в самых разных областях — от системного программирования и встроенных устройств до игр, банковских систем и искусственного интеллекта.

2. История создания языка C++

История C++ тесно связана с языком C, который был создан в 1970-х годах Деннисом Ритчи (Dennis Ritchie) в лаборатории Bell Labs.

Язык C быстро стал основой для разработки операционных систем (в том числе UNIX) и множества приложений.

Однако со временем появилась необходимость в более **структурированном и масштабируемом** подходе к программированию.

Для этого в 1979 году **Бьёрн Страуструп (Bjarne Stroustrup)**, также сотрудник Bell Labs, начал разработку расширения языка C, добавив к нему возможности **объектно-ориентированного программирования (ООП)**.

Изначально язык назывался "**C with Classes**" — «C с классами».

Позже, в 1983 году, он получил современное название **C++**, что символически означает «инкремент C» — шаг вперёд по сравнению с языком C.

3. Этапы развития и стандарты C++

Со временем язык C++ развивался, обретая новые возможности, стандарты и инструменты.

Основные версии стандарта определяются **ISO (International Organization for Standardization)**.

Версия	Год	Основные нововведения
C++98 / C++03	1998 / 2003	Первый официальный стандарт. Поддержка классов, наследования, полиморфизма, шаблонов (templates), исключений.
C++11	2011	Огромные обновления: auto, nullptr, лямбда-функции, умные указатели, многопоточность (thread), диапазонные циклы for.
C++14	2014	Небольшие улучшения синтаксиса, расширенные лямбды, литералы для чисел.
C++17	2017	std::optional, std::variant, structured bindings, улучшения для constexpr и файловой системы (filesystem).
C++20	2020	Концепции (concepts), диапазоны (ranges), корутины (coroutines), модули (modules), новые возможности шаблонов.
C++23 (последний)	2023	Улучшения стандартной библиотеки, constexpr для динамических выражений, новые утилиты и алгоритмы.

Каждый стандарт делает язык мощнее и удобнее, сохраняя при этом **обратную совместимость** — старые программы остаются рабочими.

4. Основные особенности и преимущества C++

C++ объединяет черты **низкоуровневого** и **высокоуровневого** языка программирования.

Это делает его уникальным инструментом для создания как системных, так и прикладных программ.

Ключевые особенности:

- Производительность и контроль над памятью**
 - Программист сам управляет ресурсами, указателями, памятью.
 - Это делает программы быстрыми и оптимизированными.
- Объектно-ориентированное программирование (ООП)**
 - Поддержка классов, объектов, инкапсуляции, наследования и полиморфизма.
- Шаблоны (templates)**
 - Позволяют писать обобщённый код, работающий с разными типами данных.
 - Основа для библиотеки **STL (Standard Template Library)**.

4. **STL — стандартная библиотека шаблонов**
— Включает контейнеры (vector, list, map), алгоритмы (sort, find), итераторы и др.
 5. **Совместимость с C**
— Большинство кода на C можно использовать в C++ без изменений.
 6. **Поддержка параллелизма и многопоточности**
— Начиная с C++11, встроена поддержка потоков и атомарных операций.
 7. **Кроссплатформенность**
— C++ программы компилируются под Windows, Linux, macOS, Android и другие платформы.
-

5. Сферы применения языка C++

Благодаря производительности и гибкости, C++ применяется во множестве областей:

Область	Примеры
Системное программирование	Разработка ОС, драйверов, утилит (например, ядро Windows, macOS, Linux).
Игровая индустрия	Unreal Engine, Unity (частично), CryEngine написаны на C++.
Финансовые системы	Высокопроизводительные торговые платформы, анализ данных.
Встроенные системы (Embedded)	Программирование микроконтроллеров, устройств IoT.
Научные вычисления	Моделирование, симуляции, физические и инженерные расчёты.
Приложения с GUI	Qt, wxWidgets, Visual C++ — для создания интерфейсов.
Компиляторы и языковые инструменты	GCC, Clang, MSVC — написаны на C++.

6. Структура программы на C++

Каждая программа на C++ состоит из **функций**, и **главная функция** — **main()** — точка входа в программу.

Пример простейшей программы:

```
#include <iostream> // Подключение библиотеки ввода-
вывода

using namespace std; // Использование стандартного
пространства имён

int main() { // Главная функция программы
    cout << "Hello, World!" << endl; // Вывод текста
на экран
    return 0; // Завершение программы
}
```

Разберём структуру по частям:

1. **#include <iostream>**
— Директива препроцессора. Подключает стандартную библиотеку для ввода/вывода (cin, cout).
 2. **using namespace std;**
— Указывает, что мы используем пространство имён std, чтобы обращаться к объектам без приставки std::.
 3. **int main()**
— Главная функция. С неё начинается выполнение программы. Возвращает целое значение (обычно 0 — успешное завершение).
 4. **cout << "Hello, World!" << endl;**
— Команда вывода данных в консоль.
cout — поток вывода, << — оператор перенаправления, endl — переход на новую строку.
 5. **return 0;**
— Возврат значения операционной системе, сигнализирующий успешное выполнение.
-

7. Этапы компиляции программы C++

C++ — **компилируемый язык**, что означает, что исходный код необходимо преобразовать в машинный код перед выполнением.

Процесс включает 4 основных шага:

1. **Препроцессор (Preprocessing)**
— Обрабатывает директивы #include, #define, комментарии и т. д.
2. **Компиляция (Compilation)**
— Преобразует исходный код в объектный код (.obj или .o).

3. Компоновка (Linking)

— Объединяет объектные файлы и библиотеки в один исполняемый файл.

4. Исполнение (Execution)

— Запуск программы пользователем.

8. Особенности написания и организации кода

- **Комментарии**

- // Однострочный комментарий
- /* Многострочный
комментарий */

- **Идентификаторы и имена**

— могут содержать буквы, цифры и символ `_`, но не начинаться с цифры.

- **Файловая структура**

— Код можно разделять на заголовочные файлы (`.h`) и файлы реализации (`.cpp`).

- **Пространства имён (namespaces)**

— позволяют избегать конфликтов имён в больших проектах.

9. Современная экосистема C++

- **Компиляторы:** GCC, Clang, Microsoft Visual C++, Intel C++ Compiler
 - **Среды разработки (IDE):** Visual Studio, CLion, Code::Blocks, Qt Creator
 - **Системы сборки:** CMake, Make, Meson
 - **Инструменты анализа:** Valgrind, gdb, AddressSanitizer
-

10. Заключение

C++ — фундаментальный язык программирования, который:

- объединяет низкоуровневую эффективность с высокоуровневыми концепциями;
- остаётся востребованным более 40 лет;
- лежит в основе множества современных технологий.

Изучение C++ даёт прочное понимание принципов программирования, работы памяти и алгоритмов, а также открывает путь к освоению других языков и систем.

11. Вопросы для самопроверки

1. Кто является автором языка C++ и когда он был создан?
2. Чем C++ отличается от языка C?
3. Назовите основные стандарты C++ и их отличия.
4. Какие задачи решаются с помощью C++?
5. Опишите структуру простой программы на C++.
6. Какие этапы включает процесс компиляции программы?